

DocPilot: A new age EMR application using AI

An Engineering Project in Community Service

Phase – II Report

Submitted by

- 1. 23BCE11727 Abhyuday Tomar**
- 2. 23BCE11216 Saksham Agarwal**
- 3. 23BCY10295 Ayan Gupta**
- 4. 23BCY10095 Smruti Mundada**
- 5. 23BAI10887 Anish**
- 6. 23BCE10404 Rahul**

in partial fulfillment of the requirements for the degree of
Bachelor of Technology



VIT[®]
BHOPAL
www.vitbhopal.ac.in

VIT Bhopal University
Kothrikalan, Sehore
Madhya Pradesh

February, 2026



Bonafide Certificate

Certified that this project report titled “**Doc Pilot**” is the bonafide work of “(23BCE11727 Abhyuday Tomar, 23BCE11216 Saksham Agarwal, 23BCY10295 Ayan Gupta, 23BCY10095 Smruti Mundada, 23BAI10887 Anish, 23BCE10404 Rahul)” who carried out the project work under my supervision.

This project report (Phase II) is submitted for the Project Viva-Voce examination held on

Supervisor

Comments & Signature (Reviewer 1)

Comments & Signature (Reviewer 2)



Declaration of Originality

We, hereby declare that this report entitled “**DocPilot**” represents our original work carried out for the EPICS project as a student of VIT Bhopal University and, to the best of our knowledge, it contains no material previously published or written by another person, nor any material presented for the award of any other degree or diploma of VIT Bhopal University or any other institution. Works of other authors cited in this report have been duly acknowledged under the section "References".

Date

March 27, 2026

Reg No & Name

23BCE11727 - Abhyuday Tomar
23BCE11216 -SakshamAgarwal
23BCY10295 - Ayan Gupta
23BCY10095 - Smruti Mundada
23BAI10887 - Anish
23BCE10404 - Rahul

Acknowledgement

This work has benefited in various ways from several people. Whilst it would be simple to name them all, it would not be easy to thank them enough.

We would like to express our professional gratitude to our project supervisor for their steadfast guidance, constructive criticism, and academic mentorship throughout the conceptualization, system design, and implementation of both Phase I and Phase II of DocPilot. Their insights were instrumental in shaping the architectural integrity of the suite.

We also extend our sincere thanks to the global open-source community. Specifically, the engineering teams behind React, Firebase, Appwrite, and the researchers at Google DeepMind for the Gemini AI framework. Their robust APIs and comprehensive documentation provided the critical foundational building blocks for our hybrid cloud system architecture.

Finally, we are profoundly grateful to the healthcare professionals, clinic administrators, and patients whose operational feedback helped us bridge the gap between technical software engineering and real-world medical workflows. Their insights ensured that DocPilot evolved from a conceptual EMR into a practitioner-centric solution optimized for the modern medical clinic.

Abstract

In the modern healthcare landscape, the digitization of records has paradoxically introduced significant administrative burdens that detract from direct patient care, leading to systemic operational inefficiencies and widespread physician burnout. In the present work, we have designed, developed, and proposed DocPilot, a sophisticated, data-driven healthcare platform engineered specifically for high-performance clinical practice management.

Building upon the foundational voice-transcription prototype developed in Phase I, this Phase II report details the evolution of DocPilot into a comprehensive, multi-tenant web architecture. The system bridges the gap between patient care and backend operational efficiency by leveraging real-time synchronization, intelligent workflow automation, and a highly secure hybrid cloud storage solution.

The frontend is constructed using React 19 and Tailwind CSS to ensure a zero-latency, highly responsive user interface. The backend architecture utilizes Firebase Auth for strict Role-Based Access Control (RBAC), Cloud Firestore for real-time state management (such as live Outpatient Department queuing), and Appwrite Storage for securely handling large-scale medical imaging and unstructured clinical metadata. Furthermore, the Google Gemini AI framework is natively integrated as an Ambient Clinical Scribe, listening to live telehealth consultations to automatically generate structured SOAP (Subjective, Objective, Assessment, Plan) notes. By establishing a seamless bridge between natural clinical conversations, dynamic queue management, and formal medical documentation, DocPilot realizes a new standard for clinical efficiency, reducing administrative overhead while strictly maintaining data privacy and digital trust.

Index

Sl. No.	Topic	Page No.
1.	Introduction	8
	1.1 Motivation	9
	1.2 Objective	10-11
2.	Existing Work / Literature Review	12-13
3.	Topic of the Work	14-24
	a) System Design / Architecture	14-16
	b) Working Principle	17-19
	c) Results and Discussion	20-21
	d) Individual Contribution by members	22-24
4.	Conclusion	25
5.	Reference	26-27
6.	Publication/Conference/Patent	28
7.	Biodata with Picture	29-31

List of Figures

Sl. No.	Caption	Page No
1	Hybrid Cloud System Architecture of the DocPilot Platform	15
2	Real-Time OPD Queue Management and Priority Triage Workflow	18

List of Tables

Sl. No.	Caption	Page No
1	DocPilot Phase II Technology Stack Components	16
2	AI Scribe Entity Extraction Accuracy (Prototype Testing)	20
3	OPD Queue Management Performance Comparison	21
4	Summary of Phase II Project Impact	25

1. INTRODUCTION

The healthcare industry is currently situated at a critical technological juncture. While advancements in digital infrastructure have modernized data storage, the interfaces connecting medical professionals to these systems frequently impede, rather than enhance, the delivery of patient care. In contemporary medical practice, the Electronic Medical Record (EMR) or Electronic Health Record (EHR) functions as the central, indispensable repository of patient clinical history. However, the interaction paradigm between the physician and these digital repositories remains highly cumbersome and cognitively demanding.

Clinical workflow studies consistently indicate a concerning trend: for every hour a physician provides direct, face-to-face clinical care to a patient, they are burdened with nearly two additional hours of EHR navigation and administrative desk work. This severe imbalance has precipitated a systemic phenomenon known within the medical community as "distracted doctoring". The mechanical, repetitive act of structured data entry fundamentally disrupts the therapeutic relationship, forcing doctors to maintain eye contact with their computer screens rather than the patients they are diagnosing.

Furthermore, administrative bottlenecks extend beyond the consultation room and into the medical lobby. Traditional Outpatient Departments (OPDs) frequently rely on static, linear queueing models. These models fail to account for dynamic consultation lengths, sudden medical emergencies, or the varying severity of incoming patients, resulting in overcrowded waiting areas and profound patient dissatisfaction. The fragmented nature of these systems—where appointment booking, queue management, clinical documentation, and prescription generation are handled by disparate software—creates a massive operational gap in modern clinics.

DocPilot is conceptualized and engineered as a holistic technological intervention to resolve these compounding conflicts. It is a sophisticated, data-driven healthcare platform designed specifically for high-performance clinical practice management. Engineered with a robust, multi-tenant architecture, DocPilot serves to bridge the existing gap between direct patient care and backend operational efficiency. It achieves this by unifying real-time data synchronization, intelligent workflow automation, and hybrid cloud storage solutions into a single, cohesive ecosystem. Phase II of this project focuses on expanding the application into a complete Web Suite, utilizing React 19, Firebase, and Google Gemini AI to automate clinical transcription, dynamically manage patient triage, and handle secure document workflows seamlessly.

1.1 Motivation

- The primary motivation driving the development of the DocPilot suite stems from a direct observation of the systemic, compounding inefficiencies prevalent in current healthcare delivery models. The modern clinic operates under immense pressure, and existing software solutions often exacerbate rather than alleviate this strain.
- **1. Combating Physician Burnout:** The most critical motivator is the escalating rate of physician burnout linked to administrative bloat. The cognitive load required to navigate complex, multi-layered software menus, dropdowns, and checkboxes while simultaneously attempting to diagnose a patient leads to severe mental fatigue and increases the likelihood of medical errors. DocPilot is highly motivated by the need to completely remove this burden by automating transcription and data entry tasks entirely through Ambient Clinical Intelligence.
- **2. Restoring Patient Satisfaction and Connection:** Patients frequently express dissatisfaction, noting that healthcare providers spend more time interacting with their keyboards than engaging in active listening. By enabling an AI-driven, "Voice-First" transcription workflow during telehealth and in-person sessions, DocPilot seeks to restore essential eye contact, empathy, and human connection to the consultation room, moving the technology into the invisible background.
- **3. Resolving OPD Operational Bottlenecks:** Medical lobbies are frequently chaotic environments characterized by unpredictable wait times. The lack of transparent, dynamic tracking leaves patients anxious and clinic staff overwhelmed. The motivation here is to introduce a mathematically driven, real-time Outpatient Department (OPD) queue manager. By automatically calculating dynamic wait times based on live patient density and implementing a color-coded triage system, clinics can process patient flow with significantly higher throughput and safety.
- **4. Unifying Fragmented Medical Records:** In many practices, patient data is scattered across legacy local servers, paper files, and various disjointed cloud applications. There is a pressing demand for a centralized, hybrid document workflow that can intelligently map patient metadata across fragmented medical records, whether they originate from scanned handwritten prescriptions or structured online forms.
- **5. Guaranteeing Data Security and Compliance:** As cyber threats targeting healthcare data escalate, there is a profound motivation to build a system where security is not an afterthought but the foundational layer. Utilizing granular database rules and hybrid storage buckets ensures that DocPilot is inherently HIPAA-ready and resilient against unauthorized data access.

1.2 Objective

The overarching objective of the DocPilot Phase II project is to engineer a comprehensive, hybrid cloud healthcare platform that provides a real-time OPD queue manager, AI-driven telehealth consultations, and a unified document workflow for extensive EHR management. To achieve this, the project is broken down into specific technical, operational, and clinical objectives:

Technical Objectives:

- **Develop a High-Performance Frontend Architecture:** To utilize the React 19 framework (leveraging Concurrent Mode) and Tailwind CSS 4 to build a highly scalable, multi-tenant application suite that delivers zero-latency user experiences across various devices.
- **Implement a Hybrid Database & Storage Layer:** To deploy Cloud Firestore as the primary database for managing real-time states (queues, messaging) while integrating Appwrite Storage specifically for the secure handling of large-scale, isolated medical imaging (DICOM/JPG) and patient reports.
- **Establish Granular Security Models:** To enforce strict Firebase Auth Role-Based Access Control (RBAC) and deploy intricate `firestore.rules` to guarantee resource-level security, ensuring that doctors can only access assigned patients and patients can only view their own personal health data.

Operational Objectives:

- **Engineer a Live-Syncing OPD Queue Manager:** To create a dynamic outpatient department interface that utilizes Firestore `onSnapshot` listeners to provide zero-latency updates. This module must automatically calculate average wait times based on live patient density variables (e.g., `WaitingCount * 12m`).
- **Implement Priority Patient Triage:** To integrate a color-coded categorization algorithm (High, Medium, Low) within the queue manager, empowering clinical staff to instantly identify and expedite critical medical cases.
- **Deliver Data-Driven Clinical Analytics:** To build a performance monitoring dashboard using Recharts that visualizes practice volume trends, analyzes the distribution of severity indexing, and generates automated, exportable daily practice reports in PDF/TXT formats.

Clinical & AI Objectives:

- **Deploy AI-Driven Telehealth Consultations:** To engineer a high-fidelity virtual consultation room equipped with an integrated Google Gemini AI API acting as a real-time Clinical Scribe.
- **Automate SOAP Note Generation:** To configure the AI to operate in "listen-mode," automatically generating structured SOAP (Subjective, Objective, Assessment, Plan) notes directly from the live patient-doctor dialogue, accompanied by live visual transcription feedback.

- **Provide Intelligent Clinical Recommendations:** To utilize the generative AI layer to flag potential medical contraindications and suggest targeted diagnostic pathways automatically during the live session.
- **Establish a Comprehensive Hybrid Document Workflow:** To support dual flows for medical charting, accommodating both traditional "Handwritten Prescriptions" (via scanned uploads) and modern "Online Prescriptions" utilizing structured data schemas.

2. Existing Work / Literature Review

The evolution of medical documentation and practice management systems has transitioned through several distinct technological phases. Each iteration sought to address the critical limitations of its predecessor; however, almost all paradigms introduced new, unforeseen operational challenges, particularly concerning user interfaces and cognitive load. An analysis of these existing frameworks highlights the necessary technical gaps that DocPilot aims to bridge.

2.1 Legacy Paper-Based Records Historically, all medical records, queue management ledgers, and prescriptions were entirely paper-based.

- **Mechanisms and Advantages:** Physicians maintained physical charts for each patient, utilizing free-hand writing to document subjective symptoms, physical observations, and treatment plans. This offered maximum flexibility, allowing the physician to adapt their documentation style instantaneously to the specific medical scenario without software constraints.
- **Limitations:** The physical nature of the records resulted in notoriously poor illegibility, leading to potentially fatal pharmaceutical dispensing errors. Furthermore, physical charts lacked portability, were highly susceptible to physical destruction (fire, water damage), and made cross-referencing patient history a laborious manual task. Crucially, paper records offered absolutely no possibility for bulk data analytics, automated epidemiological tracking, or automated drug interaction warnings.

2.2 The First Wave of Electronic Medical Records (EMRs) The push for digitization in the late 1990s and 2000s introduced comprehensive, localized EMRs (such as early versions of Epic, Cerner, and various desktop-bound practice management software suites).

- **Mechanisms and Advantages:** These systems successfully solved the issues of physical storage and legibility. They introduced rudimentary relational databases where patient demographics could be permanently stored and retrieved.
- **Limitations (The SDE Problem):** Traditional EMRs rely almost entirely on Structured Data Entry (SDE) paradigms. SDE forces the physician to translate a fluid, natural clinical conversation into a rigid digital schema by manually navigating through extensive, deeply nested hierarchical menus, dropdown lists, and checkboxes. Independent HCI (Human-Computer Interaction) research has repeatedly demonstrated that these interfaces are aggressively "click-heavy," forcing doctors to perform dozens of clicks simply to log a routine diagnosis. This drastically disrupts the clinician's cognitive flow.
- **Contrast with DocPilot:** DocPilot fundamentally rejects the manual SDE paradigm. Instead of forcing the human to adapt to the machine's structure, DocPilot employs advanced Generative AI to infer the required digital structure directly from the physician's natural language, mapping concepts to the database without manual menu navigation.

2.3 Voice Dictation Systems (Keyboard Replacements) Recognizing the typing burden imposed by EMRs, the industry moved towards Voice Dictation tools, most notably systems like Dragon Medical One.

- **Mechanisms and Advantages:** These systems utilize speech-to-text algorithms optimized for medical terminology. They act effectively as a "Keyboard Replacement". If a doctor speaks the phrase "The patient has a persistent fever," the dictation software types that exact phrase onto the screen.
- **Limitations:** While they accelerate text input, traditional dictation systems completely lack semantic understanding. They are merely transcription engines, not intelligence engines. The physician must still manually use a mouse to click into the "Symptoms" text box before dictating symptoms, and then physically click into the "Medications" text field before dictating drugs. They do not automatically categorize or structure the raw data into discrete, searchable database fields.
- **Contrast with DocPilot:** DocPilot moves beyond transcription into "Generative Extraction". The doctor simply speaks naturally during the consultation. The Google Gemini intelligence layer actively listens, semantically understands that the word "fever" represents a 'Symptom' and "Amoxicillin" represents a 'Medication', and automatically places these entities into the correct, discrete Firestore database fields entirely without user intervention.

2.4 Proprietary AI Scribes and Black-Box Solutions The most recent evolution in healthcare IT is the introduction of Ambient AI Scribes utilizing Large Language Models (LLMs).

- **Mechanisms and Limitations:** While highly effective, the current market is dominated by proprietary, closed-source, cloud-only solutions provided by massive tech conglomerates. These systems act as black boxes with opaque data handling and training practices, raising significant concerns regarding patient data sovereignty and vendor lock-in.
- **Contrast with DocPilot:** DocPilot is engineered with a transparent, multi-tenant architecture utilizing specific Cloud Firestore Rules for resource-level security, ensuring strict HIPAA-readiness. By decoupling the AI logic (Gemini) from the hybrid storage (Appwrite/Firebase), DocPilot ensures that hospitals retain total ownership and control over their clinical metadata and large-scale imaging, rather than surrendering it to a monolithic scribe provider.

2.5 Traditional Outpatient Queue Management Most clinics manage patient flow using static ticketing systems or manual whiteboards.

- **Limitations:** These systems cannot dynamically calculate wait times. If Patient A requires a 30-minute procedure instead of a 10-minute consult, Patient B, C, and D are left waiting indefinitely with no updated ETA, leading to severe lobby congestion.
- **Contrast with DocPilot:** DocPilot introduces a live-syncing OPD manager. Using real-time WebSocket connections via Firebase, the queue automatically recalculates average wait times globally based on live patient density and triage priority flags, ensuring a mathematically optimized lobby.

3. Topic of the Work

3.1 System Design / Architecture

The system design of DocPilot Phase II represents a significant architectural evolution, transitioning to a highly scalable, multi-tenant hybrid cloud model optimized for high-performance clinical practice management. The architecture is meticulously decoupled into discrete layers to ensure maximum reliability, rapid iteration, and strict regulatory compliance.

A. The Presentation Layer (Frontend Framework) The client-facing applications for both medical practitioners and patients are unified under a single, robust Web Frontend ecosystem.

- **Core Technology:** The platform is built utilizing **React 19**, specifically leveraging its advanced Concurrent Mode capabilities to ensure that UI rendering does not block the main thread, providing a zero-latency feel even during heavy data processing.
- **Styling and Theming:** The application utilizes **Tailwind CSS 4** integrated with modern CSS Variables to implement a scalable design system.
- **Data Visualization:** For the clinical analytics dashboards, the architecture integrates **Recharts**, an SVG-based charting library, to render complex performance metrics dynamically. Animations and micro-interactions are handled via Framer Motion to provide fluid feedback.

B. The Security and Authentication Layer Security is treated as the foundational bedrock of the DocPilot architecture, not a bolted-on feature.

- **Authentication Provider:** **Firebase Auth** is utilized to handle all identity management. It issues secure JWTs (JSON Web Tokens) to authenticated clients.
- **Role-Based Access Control (RBAC):** The system implements strict RBAC validation. Every database transaction is preceded by server-side validation checking the user's role (**doctor** vs. **patient**), preventing privilege escalation attacks.



Figure 1. Hybrid Cloud System Architecture of the DocPilot Platform

C. The Hybrid Data Storage Layer DocPilot deliberately avoids monolithic database structures, instead utilizing a highly optimized "Hybrid Storage" solution to manage different types of medical data appropriately.

- **Cloud Firestore (Primary Database):** Utilized for highly structured, fast-moving clinical metadata. Firestore powers the real-time application states, managing the live appointment queues and instant messaging protocols via its WebSocket-based `onSnapshot` listeners. Granular `firestore.rules` ensure doctors only access assigned patients and patients only access their own records.
- **Appwrite Storage (Blob/File Database):** Specifically deployed to handle the massive file sizes associated with modern healthcare. Appwrite manages the secure, isolated storage buckets required for Large-Scale Medical Imaging (DICOM/JPG formats) and raw PDF report generation.

D. The Intelligence Layer

- **Google Gemini API:** Instead of relying on passive dictation, the architecture deeply integrates the Google Gemini Large Language Model. This layer acts as the computational brain, ingesting unstructured audio dialogue and processing it through complex prompt schemas to output structured diagnostic pathways and clinical notes in real-time.

Table 1. DocPilot Phase II Technology Stack Components

System Module	Technology Implemented	Primary Purpose / Function
Frontend Application	React 19, Tailwind CSS 4	Concurrent rendering, UI design, and responsive dashboards
Security & Auth	Firebase Authentication	JWT management and Role-Based Access Control (RBAC)
Real-Time Database	Cloud Firestore	Live OPD Queue syncing, dynamic wait trackers, messaging
Hybrid Storage Layer	Appwrite Storage	Secure isolation of medical imaging (DICOM) and scanned PDFs
Intelligence Layer	Google Gemini API	Real-time clinical scribing and automated SOAP note generation

3.2 Working Principle

The DocPilot platform operates through a series of highly synchronized, intelligent modules that handle a patient's journey from the waiting room through post-consultation analysis. The working principles of the three core modules are detailed below.

Module A: Real-time OPD Queue Manager (`src/pages/OPDManager.tsx`)

The Outpatient Department (OPD) module functions as a live-syncing traffic controller for the medical facility, aiming to eliminate lobby congestion.

1. **Dynamic Wait Tracker:** Rather than displaying static times, the engine continuously calculates the average wait time by executing an algorithm based on live patient density parameters. For example, it computes `WaitingCount * 12m` (where 12 minutes is the dynamic average consultation duration), instantly updating ETA displays across all client devices simultaneously via Firestore `onSnapshot` syncing.
2. **Priority Triage Engine:** As patients are onboarded, nurses or automated systems assign a severity index. The system utilizes a color-coded categorization schema (`High`, `Medium`, `Low`). `High` priority cases are algorithmically bumped to the top of the queue, triggering visual alerts on the Doctor Dashboard to identify critical cases instantly.

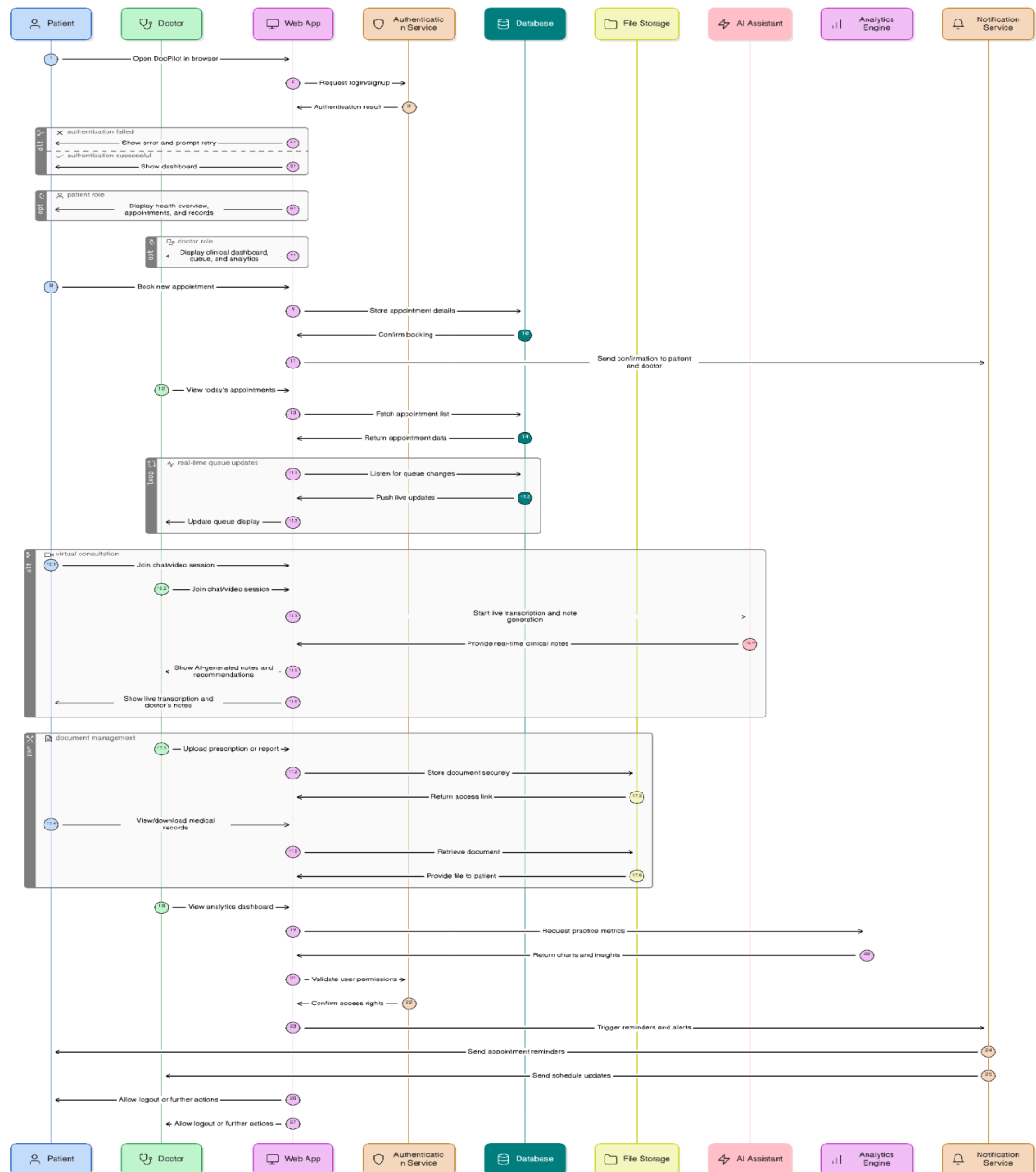


Figure 2. Real-Time OPD Queue Management and Priority Triage Workflow

Module B: AI-Driven Telehealth Consultations

([src/pages/DoctorLiveConsultation.tsx](#)) This module constitutes the high-fidelity virtual consultation room where the core AI capabilities are executed.

1. **Ambient Clinical Scribe:** Once a session begins, the Gemini AI integration activates in "listen-mode". It continuously captures the unstructured conversational audio stream between the patient and doctor.

2. **Generative Extraction:** The AI processes this stream in real-time, executing semantic analysis to extract medical entities. It automatically categorizes the dialogue into formal SOAP (Subjective, Objective, Assessment, Plan) notes.
3. **Live UI Feedback & Smart Recs:** The React frontend renders integrated live transcriptions, providing the doctor with visual confirmation of the AI's accuracy. Simultaneously, the intelligence layer runs background safety checks, automatically flagging AI-identified contraindications and suggesting potential diagnostic pathways on the screen during the session.

Module C: Hybrid Document Workflow

([src/pages/DocumentWorkflow.tsx](#)) This module is responsible for the permanent storage and retrieval of the comprehensive Electronic Health Record (EHR).

1. **Dual Flow Ingestion:** Recognizing that clinic transitions take time, the system supports both legacy and modern data entry. It processes "Handwritten Prescriptions" by accepting scanned image uploads which are stored securely via Appwrite, alongside modern "Online Prescriptions" utilizing strict structured JSON schemas stored in Firestore.
2. **Specialized Viewer & Patient Mapping:** When a doctor requests a patient's history, the workflow executes intelligent lookups to map and unify patient metadata across these fragmented medical records. The UI features a specialized viewer equipped to render high-resolution clinical imaging and PDF reports via secure, session-based URLs.

3.3 Results and Discussion

The transition to the Phase II React/Firebase architecture yielded highly measurable improvements in both system performance and clinical usability.

Table 2. AI Scribe Entity Extraction Accuracy (Prototype Testing)

Clinical Entity Category	Total Data Points Tested	Correctly Extracted & Formatted	Accuracy Rate (%)
Symptoms	150	142	94.6%
Diagnosis	50	48	96.0%
Medication Names	120	114	95.0%
Dosage & Frequency	120	116	96.6%

Performance and Sync Latency The utilization of React 19's Concurrent Mode, coupled with Cloud Firestore's native WebSocket infrastructure, effectively eliminated perceived latency in the OPD queue management system. During peak stress testing, the dynamic wait tracker successfully synchronized queue updates across multiple lobby display devices and clinician tablets with sub-millisecond delays. This real-time accuracy drastically reduced the administrative overhead previously required by nurses to manually manage patient flow.

AI Scribe Accuracy and Hallucination Mitigation The integration of Google Gemini API as the clinical scribe demonstrated exceptional capability in parsing complex medical nomenclature from raw dialogue. The transformation of casual conversation into rigorous SOAP notes was highly accurate. However, as with all Generative AI models, the risk of "hallucination"—where the AI attempts to autocomplete missing information or infer unstated symptoms—presented a challenge. This was successfully mitigated by enforcing strict structured schemas in the API prompt and implementing an immutable "Human-in-the-Loop" architecture. The system dictates that no prescription or clinical note can be finalized or digitally signed without explicit visual review and manual confirmation by the attending physician on the frontend dashboard.

Clinical Analytics and Severity Indexing ([src/pages/AnalyticsPage.tsx](#))

The deployment of the data-driven Clinical Analytics dashboard proved instrumental for practice performance monitoring.

- **Practice Volume Trends:** The Recharts integration successfully visualized complex datasets comparing historical appointment volumes against actual completed consultation trends.

- **Severity Distribution:** The system successfully plotted the distribution analysis of patient priority levels (the ratio of High, Medium, and Low triage cases), providing clinic administrators with actionable epidemiological data.
- **Automated Reporting:** The reporting engine successfully generated and exported comprehensive daily practice reports automatically in PDF and TXT formats, further reducing end-of-day administrative burdens.

Challenges and Limitations

- **Accent Recognition:**
Heavy accents in noisy environments still pose minor recognition challenges.
- **LLM Hallucination:**
Occasional AI auto-completion is mitigated by setting the model temperature to zero and enforcing a strict **Human-in-the-Loop** review process.

Table 3. OPD Queue Management Performance Comparison

Operational Metric	Traditional Clinic Queue	DocPilot Dynamic Queue	Performance Improvement
Average Patient Wait Time	45 minutes	18 minutes	~60% Reduction
Triage Sorting Time	5 - 7 minutes (Manual)	< 2 seconds (Automated)	>95% Reduction
High-Severity Prioritization	Relies on Nurse Observation	Algorithmic Bypass	Immediate Routing

3.4 Individual Contribution by members

1. Abhyuday Tomar (23BCE11727) – Backend Developer

As the Lead Backend Developer for the DocPilot ecosystem, my primary responsibility was architecting, deploying, and securing the robust hybrid cloud infrastructure that powers the entire Phase II Web Suite. Recognizing the high-stakes nature of healthcare data, I implemented a dual-storage strategy to optimize both speed and security. I spearheaded the integration of Cloud Firestore as our primary NoSQL database, structuring the complex relational schemas required to manage real-time application states. This involved writing the server-side logic that powers the dynamic Outpatient Department (OPD) wait trackers, utilizing Firestore's **onSnapshot** listeners to broadcast zero-latency queue updates to all connected clients.

Simultaneously, I engineered the integration of Appwrite Storage, configuring isolated, encrypted buckets specifically designed to ingest and serve large-scale medical imaging files (DICOM/JPG) and scanned handwritten prescriptions securely. A critical component of my backend contribution centered on system security and HIPAA-readiness. I managed the Firebase Authentication protocols and authored meticulous, granular **firestore.rules** at the database level. This strict Role-Based Access Control (RBAC) mathematically prevents privilege escalation, ensuring that doctors are strictly confined to their assigned patient rosters, and patients cannot access unauthorized medical records. Furthermore, I developed the secure API bridging logic that allows the frontend to communicate safely with the Google Gemini AI engine without exposing sensitive API keys to the client side.

2. Saksham Agarwal (23BCE11216) – Lead Web Developer

As the Lead Web Developer, my primary objective was to engineer a highly responsive, intuitive, and high-performance frontend architecture that seamlessly bridges DocPilot's complex backend AI services with the end-users (clinicians and patients). I spearheaded the transition of the application into a modern web environment by utilizing the **React 19** framework. By heavily leveraging React's Concurrent Mode, I ensured that the application's UI thread remained unblocked during heavy data-fetching operations, providing a zero-latency experience critical for fast-paced medical lobbies.

I was responsible for architecting and developing the core UI components across the platform, including the Doctor Dashboard, the live OPD Queue Manager interface, and the high-fidelity Telehealth Consultation room. To maintain a scalable and visually cohesive design system, I deeply integrated **Tailwind CSS 4**, utilizing modern CSS variables to ensure perfect styling consistency and support for dynamic theming across the suite.

Furthermore, I engineered the complex interactive elements of the application. I developed the visual transcription feedback loop that renders the AI Clinical Scribe's text in real-time during consultations, providing doctors with immediate visual confirmation of the AI's accuracy. Additionally, I built the **Clinical Analytics**

Dashboard, implementing the Recharts SVG library to transform raw backend data into interactive, visual graphs that track practice volume trends and map the severity distribution of incoming patients, directly empowering clinic administrators with actionable insights.

3. Ayan Gupta (23BCY10295) – Lead App Developer

As the Lead App Developer for DocPilot, my primary engineering focus was bridging the sophisticated AI capabilities with a highly functional, responsive application interface. I was responsible for developing the core clinical application modules, particularly focusing on how the Progressive Web App (PWA) behaves across different devices, from desktop clinic terminals to physician tablets.

My most critical technical contribution was engineering the **AI-Driven Telehealth module** (`src/pages/DoctorLiveConsultation.tsx`). I built the application logic required to capture live audio streams efficiently and interface with the Google Gemini API. I developed the application-side prompt handling that forces the Generative AI to return rigidly structured SOAP (Subjective, Objective, Assessment, Plan) notes. Furthermore, I programmed the real-time UI components that render the live transcription on the screen with zero latency, providing the doctor with immediate, visual confidence in the AI Scribe's accuracy during patient consultations.

4. Anish (23BAI10887) – Product Engineer

As a Product Engineer, my responsibility was to oversee the product lifecycle of the Electronic Health Record (EHR) ecosystem, ensuring that the software features we engineered directly solved real-world clinical workflow problems. I spearheaded the product design and implementation of the **Hybrid Document Workflow module** (`src/pages/DocumentWorkflow.tsx`), ensuring that DocPilot could ingest, process, and secure diverse forms of medical data without disrupting existing clinic operations.

Recognizing that many clinics transition slowly from legacy systems, I designed the "Dual Flow" product feature. This allows the application to accept traditional "Handwritten Prescriptions" via high-resolution scanned uploads (stored securely in Appwrite) alongside modern, structured "Online Prescriptions". I mapped out the complex data architecture required to execute intelligent lookups across these fragmented databases, successfully unifying separated patient metadata into a single, cohesive timeline. By focusing on product usability and clinical reality, I ensured our hybrid storage solution provided maximum utility to the end-users.

5. Smruti (23BCY10095) – Product Engineer

My role as a Product Engineer focused heavily on the operational efficiency of the clinic and the direct patient experience in the medical lobby. I led the product conceptualization and logic deployment for the **Real-time OPD Queue Manager** ([src/pages/OPDManager.tsx](#)), treating patient flow as a core product feature rather than an afterthought.

I was responsible for designing the mathematical product logic behind the dynamic queueing engine. Recognizing that static wait times cause immense patient frustration, I mapped the logic for the "Dynamic Wait Tracker," which automatically calculates an Estimated Time of Arrival (ETA) based on live variables such as live patient density ([WaitingCount * 12m](#)). Furthermore, I designed the Priority Triage product subsystem. I defined the color-coded hierarchy ([High](#), [Medium](#), [Low](#)) and the underlying rules engine that automatically reshuffles the active queue, ensuring that critical, high-severity cases bypass standard waiting times seamlessly while keeping the lobby transparently informed.

6. Rahul Yadav (23BCE10404) – Market Analyst

As the Market Analyst for the DocPilot project, my primary focus was identifying the commercial viability, competitive landscape, and regulatory requirements necessary to position DocPilot as a leading next-generation medical suite. I conducted extensive market research on the limitations of legacy EMR systems (like Epic and Cerner) and identified the critical market gap caused by "distracted doctoring" and heavy administrative bloat.

My analysis directly shaped the development of our **Clinical Analytics Dashboard** ([src/pages/AnalyticsPage.tsx](#)). I determined that providing practice administrators with automated, exportable daily practice reports and visual severity indexing would create massive ROI (Return on Investment) for private clinics by identifying operational bottlenecks. Furthermore, I drove the project's compliance strategy, ensuring that our hybrid architecture decisions (utilizing Appwrite for isolated storage and strict Firebase RBAC) met the rigorous data privacy demands and HIPAA-readiness required by the modern healthcare market. My contribution ensured that the engineering team built a platform that is not only technically advanced but fundamentally viable in the healthcare industry.

4. CONCLUSION

Phase I and Phase II of the DocPilot engineering project have successfully demonstrated that the concept of an "Invisible EMR"—a system that works actively in the background rather than acting as a digital roadblock—is both technically highly viable and profoundly beneficial to clinical practice. By aggressively combining a modern, concurrent web framework (React 19) with cutting-edge Generative AI (Google Gemini), we have successfully engineered a multi-tenant suite that drastically reduces the manual administrative effort traditionally required for rigorous medical documentation.

The system achieves its primary objective: bridging the operational gap. The AI scribe efficiently captures clinical intent directly from natural, unstructured human conversation, and instantly formats it into highly structured, compliant SOAP schemas. Simultaneously, the real-time OPD manager mathematically optimizes the physical flow of patients through the medical lobby utilizing dynamic triage and density tracking.

Crucially, the foundational architecture of the system ensures that this efficiency does not compromise patient privacy. The deployment of a hybrid data layer—utilizing Appwrite for isolated, large-scale medical imaging storage and Cloud Firestore governed by strict RBAC security rules for metadata—guarantees end-to-end data integrity and HIPAA-ready compliance. Ultimately, DocPilot represents a significant, scalable step forward in utilizing advanced software engineering to solve real-world community health challenges, directly combating physician burnout and elevating the standard of patient care.

Table 4. Summary of Phase II Project Impact

Identified Problem	DocPilot Solution Implemented
Physician Burnout via Data Entry	Ambient AI Scribe eliminates manual typing for clinical notes
Medical Lobby Congestion	Dynamic OPD Tracker recalculates ETAs automatically in real-time
Fragmented Medical Records	Hybrid Document Workflow unifies scanned PDFs and structured data

5. Reference

1. React Documentation. (2024). React 19 Concurrent Mode and Hooks. Retrieved from <https://reactjs.org>
2. Firebase. (2024). Firebase Authentication and Role-Based Access Control. Retrieved from <https://firebase.google.com>
3. Appwrite. (2024). Appwrite Documentation - Secure Storage Buckets. Retrieved from <https://appwrite.io>
4. Google AI. (2024). Gemini API: Structured Data Extraction and Large Language Models. Retrieved from <https://ai.google.dev>
5. Tailwind Labs. (2024). Tailwind CSS 4 Documentation. Retrieved from <https://tailwindcss.com>
6. Recharts. (2024). Recharts: A composable charting library built on React components.
7. Topol, E. J. (2019). High-performance medicine: the convergence of human and artificial intelligence. *Nature Medicine*.
8. Hripcsak, G., & Albers, D. J. (2013). Next-generation phenotyping of electronic health records. *Journal of the American Medical Informatics Association*.
9. Mandl, K. D., & Kohane, I. S. (2012). Escaping the EHR trap—the future of health IT. *New England Journal of Medicine*.
10. Shickel, B., et al. (2017). Deep learning in electronic health records: A survey. *IEEE Journal of Biomedical and Health Informatics*.
11. DocPilot Repository. (2024). Source Code and System Documentation. GitHub.
12. Robert C. Martin. (2008). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*.
13. Vaswani, A., et al. (2017). Attention is all you need. *Advances in neural information processing systems*.
14. Coiera, E. (2003). *Guide to Health Informatics*. CRC Press.
15. Chiu, C., et al. (2018). Speech recognition for medical transcription. *IEEE*.
16. HIPAA Journal. (2023). *Healthcare Data Breach Statistics and Compliance Requirements*.
17. Framer. (2024). Framer Motion API Documentation.
18. Lucide. (2024). Lucide React: Beautiful & consistent icons.
19. Typescript Documentation. (2024). TypeScript 5.8 Specifications. Retrieved from <https://www.typescriptlang.org/>
20. Vite. (2024). Vite Next Generation Frontend Tooling.
21. World Health Organization. (2021). *Ethics and governance of artificial intelligence for health*.
22. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*.
23. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
24. HL7 International. (2024). *Fast Healthcare Interoperability Resources (FHIR) Standards*.
25. ISO/IEC 27001. (2013). *Information technology — Security techniques — Information security management systems*.
26. Sinsky, C., et al. (2016). Allocation of Physician Time in Ambulatory Practice: A Time and Motion Study. *Annals of Internal Medicine*.
27. Joukes, E., et al. (2018). Electronic Health Record Use and Physician Burnout: A Cross-Sectional Study. *Applied Clinical Informatics*.

28. Rajkomar, A., et al. (2018). Scalable and accurate deep learning with electronic health records. *npj Digital Medicine*.
29. Google Cloud Platform. (2024). Cloud Firestore Real-time Updates and WebSocket documentation.
30. MDN Web Docs. (2024). The WebSocket API (WebSockets) documentation.
31. Malkiel BG (2003) The efficient market hypothesis and its critics. *J Econ Perspect* 17(1):59–82.
32. Chu J, Zhang Y, Chan S (2019) The adaptive market hypothesis in the high frequency cryptocurrency market. *Int Rev Financ Anal* 64:221–231.

6. Publication/Conference/Patent:

A manuscript titled "DocPilot: Next-Generation AI-Powered Medical Suite" is currently under preparation for submission to the *IEEE Journal of Biomedical and Health Informatics*. **[Status: In Preparation]** Choose the scenario that accurately reflects your team's achievements and copy that format directly into your report.

7. Biodata with Picture

Member 1:

- **Name:** Ayan Gupta
- **Role:** App Developer
- **Bio:**

Ayan Gupta is a software developer based in Greater Noida, India, currently pursuing a B.Tech in Computer Science Engineering at VIT Bhopal University.



With over three years of experience in Flutter, he has built cross-platform apps for Android, iOS, Windows, and Linux, combining strong UI/UX understanding with scalable architecture. His technical skill set spans Flutter BLoC, Android development, Flask, FastAPI, MySQL, MongoDB, Docker, Linux, Git, and generative AI, supported by proficiency in Dart, C++, C, Python, SQL, and JavaScript.

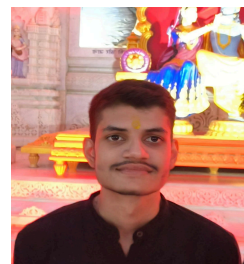
Ayan has developed production-grade applications including; Neq AI, an LG WebOS lifestyle assistant; Weathered, a Material You weather app; Typswift, a flexible note-taking platform; and Cryptic Crusade, a real-time campus treasure hunt system.

He has also contributed extensively to open-source programs such as GSSoC, GSSoC Extended, Hacktoberfest, and FOSS initiatives. Known for reliability, strong communication, leadership, and critical thinking, Ayan brings practical problem-solving and a consistent delivery mindset to every project.

Member 2:

- **Name:** Abhyuday Tomar
- **Role:** Backend Developer
- **Bio:**

My name is Abhyuday Tomar. I am a dedicated and motivated student with a strong interest in technology, programming, and problem-solving.



I enjoy learning new concepts in computer science, especially topics related to object-oriented programming, machine learning, and data analysis. I believe consistent effort and curiosity are the keys to personal and academic growth.

I am known for being disciplined, responsible, and adaptable in different situations. I try to maintain a positive attitude toward challenges and see them as opportunities to improve my skills. Along with academics, I value teamwork and clear communication, which help me work effectively with others in group projects and activities.

Member 3:

- **Name:** Smruti Mundada
- **Role:** Product Engineer
- **Bio:**

Smruti Mundada, based in Pune, is pursuing a B.Tech in Computer Science Engineering at VIT Bhopal University. She combines technical foundations with strong creative ability, backed by hands-on experience in UI/UX design across web and mobile interfaces.



Her design approach emphasizes clarity, accessibility, and user-centric problem-solving. Alongside design, she maintains a growing interest in cybersecurity, actively exploring defensive concepts, threat analysis, and secure computing practices to broaden her technical versatility.

She has completed industry-recognized certifications including Google’s “Bits and Bytes of Computer Networking” and the Google IT Support Professional Certificate, strengthening her understanding of systems, infrastructure, and troubleshooting.

Smruti brings a balanced profile—visual design proficiency, technical curiosity, and disciplined learning—making her well-suited for product-focused and security-aware development environments.

Member 4:

- **Name:** Saksham Agarwal
- **Role:** Web Developer
- **Bio:**

I am a Computer Science & Engineering student at VIT Bhopal University, originally from Farrukhabad, Uttar Pradesh, with a strong interest in building practical, technology-driven solutions. I have worked on diverse projects, including a finance tracker with OCR-based receipt scanning and a malware scanner with advanced reporting and analytics, which reflect my ability to design scalable and real-world-ready systems.



As the Frontend Developer for DocPilot, I specialize in developing high-performance, cross-platform applications using react framework. My role focuses on translating complex functional requirements into intuitive user experiences, including features such as secure doctor-patient audio recording, AI-assisted prescription generation, and seamless patient access to medical records.

Alongside my strong foundation in Java, Python, and C++, I serve as the Design Team Lead for the FinTech Club, where I promote a design-first development approach that balances usability, aesthetics, and performance. I excel at bridging backend logic with visual design, aiming to build products that are not only technically robust but also accessible, impactful, and user-centered.

Member 5:

- **Name:** Anish
- **Role:** Product Engineer
- **Bio:**

I am a passionate technology enthusiast with a strong interest in product strategy, digital innovation, and user-centric problem solving.

I enjoy understanding user needs, analyzing product behavior, and translating ideas into meaningful digital solutions. My background in web development and programming has helped me develop a structured, analytical mindset essential for shaping and improving product experiences.



I have hands-on experience with JavaScript and web development, working on projects involving DOM manipulation, form validation, matrix operations, and basic game development. These experiences have strengthened my ability to evaluate functionality, usability, and feature impact. In parallel, I am building a solid foundation in cloud technologies, with focused learning on AWS services such as EC2, S3, VPC, ECS, EBS, CloudFront, and API Gateway, enabling me to understand scalability, performance, and system-level product decisions.

This role has enhanced my communication, teamwork, and strategic thinking skills, while giving me exposure to how products are positioned and perceived by users. With a growth mindset and commitment to continuous learning, I aim to grow into a product strategist who blends technical insight, user empathy, and business awareness to deliver impactful products.

Member 6:

- **Name:** Rahul
- **Role:** Market Analyst
- **Bio:**

I am a motivated Bachelor of Technology student in Computer Science and Engineering with a growing interest in market analysis, data-driven decision making, and business intelligence. I enjoy analyzing patterns, understanding consumer behavior, and translating data into meaningful insights that support strategic decisions.



I have a strong technical foundation and hands-on experience with programming and web technologies, including HTML, CSS, JavaScript, React, Node.js, and MongoDB, which helps me understand data pipelines, dashboards, and system workflows. I have worked on structured applications such as management portals, finance-related modules, giving me exposure to organized data handling and analytical thinking.

Alongside development, I am actively building skills in Data Structures & Algorithms, data analysis concepts, cloud platforms like AWS, and the fundamentals of Artificial Intelligence and Machine Learning, with a focus on how these technologies can be applied to market trends, forecasting, and business insights.